Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

навчальної дисципліни

ТЕХНОЛОГІЇ ПРОГРАМУВАННЯ МОВОЮ JAVA

ФКЗЕ. 1234ТПЈ16. 5ЛР

Студент NeYurii

Група GROUP-NUMBER

Перевірив Lecturer

2026 - 2027

Тема: Застосування успадкування при створенні об’єктно-орієнтованих додатків мовою Java.

Мета: Навчитися визначати особливості успадкування у мові Java та використовувати їх при створенні додатків мовою Java.

Обладнання: Текстовий редактор, OpenJDK.

Хід роботи

1 Для здобуття мінімально необхідного похідного балу за виконання лабораторної роботи з таблиці 1 обрати завдання до виконання. Номер варіанту за списком для кожної підгрупи.

Таблиця 1 - Індивідуальне завдання

Варіант	**Завдання**
16	Ієрархія має містити класи: “Рослина”, “Квітка”, “Кущ” та “Дерево”. Класи мусять мати поля та методи, що описують різні типи рослин, як живі істоти, що мають певні морфологічні особливості та належать до певного виду. Класи мусять мати всі необхідні конструктори, set- та get-методи, та метод, що відбиває зростання конкретної рослини (інформацію про зростання треба друкувати на екрані).

2 Для здобуття більш високого балу за виконання лабораторної роботи виконати додатково одне завдання (рівень 1) або два завдання (рівень 2) з таблиці 2.

Таблиця 2 - Ускладнене завдання

Варіант	**Завдання**
16	Створіть клас Student ,що містить поля Прізвище, Ім’я, Група, Рік навчання, Мінімальний розмір стипендії, Рейтинговий бал. Від класу Student створити ієрархію успадкування– Бакалавр, Магістр, Аспірант. В класах Магістр та Аспірант передбачити поля Наявність наукових робіт та Наявність бюджетного місця. Створити метод getScholarship() для класу Student, який повертає суму стипендії. Розрахунок суми стипендії залежить від Рейтингового балу, а саме: - 0 – 60: мінімальний - 61 – 74: мінімальний * 0,2 - 75 – 89: мінімальний * 0,35 - 90 – 100: мінімальний * 0,5 Передбачити для класів Бакалавр, Магістр, Аспірант різний мінімальний розмір стипендії. Для класів Магістр та Аспірант підвищити рейтинговий бал на 10% при наявності наукових робіт. Створити масив типу Student, що містить об’єкти класів Бакалавр, Магістр та Aspirant. Викликати метод getScholarship() для кожного елемента масиву. Вивести результати.

3 Створити та надати викладачеві звіт з виконання лабораторної роботи. Звіт повинен містити тему, мету та обладнання для виконання лабораторної роботи,

короткий опис постанови задачи для кожного з завдань, лістинг програми, скріншоти результатів тестування програми.

Лістинг:

Перше завдання:

```
public class Task1 {
        public static void main(String[] args)
        {
                Flower flower = new Flower();
                Bush bush = new Bush();
                Tree tree = new Tree();

                Plant[] plants = {flower, bush, tree};

                for (int i = 0; i < 10; ++i) {
                        System.out.println("\n" + i * 100 + " days:");
                        for (Plant p : plants) {
                                p.printInfo();
                                p.grow(100);
                        }
                }
        }
}

class Plant {
        protected int age_; // in days
        protected double height_; // in meters
        protected boolean dead_ = false;

        Plant() { this(0, 0); }
        Plant(int age, double height)
        {
                this.age_ = age;
                this.height_ = height;
        }

        void grow(int daysPassed) { this.age_ += daysPassed; }
        void printInfo()
        {
                System.out.println("Age: " + this.age_ +
                                   " days, Height: " + this.height_ +
                                   " meters, " + (this.dead_ ? "Dead" :
        "Alive"));
        }

        // Getters
        int age()      { return this.age_; }
        double height() { return this.height_; }
        boolean dead() { return this.dead_; }

        // Setters
        void age(int age)          { this.age_ = Math.max(age, 0); }
        void height(double height) { this.height_ = Math.max(height, 0); }
        void dead(boolean dead)    { this.dead_ = dead; }
}
```

```
class Flower extends Plant {
        private static final double HEIGHT_MIN = 0.01;
        private static final double HEIGHT_LIMIT = 0.3;
        private static final int AGE_LIMIT = 200; // One-seasoned flower
        private static final int GROWTH_AGE_LIMIT = 120;

        protected String color_;

        Flower() { this(0, HEIGHT_MIN, "Red"); }

        Flower(int age, double height, String color)
        {
                super(age, height);
                this.color_ = color;
        }

        @Override
        void grow(int daysPassed)
        {
                super.grow(daysPassed);

                super.height_ = HEIGHT_MIN + (HEIGHT_LIMIT - HEIGHT_MIN) *
                                             ((double)Math.min(super.age_,
        GROWTH_AGE_LIMIT) / GROWTH_AGE_LIMIT);

                super.dead_ = (super.age_ > AGE_LIMIT);
                if (super.dead_) this.color_ = "Rotten gray";
        }

        @Override
        void printInfo()
        {
                System.out.print("Flower of " + this.color_ + " color, ");
                super.printInfo();
        }

        String color() { return this.color_; }
        void color(String color) { this.color_ = color; }

        @Override
        void height(double height)
        {
                this.height_ = Math.clamp(height, HEIGHT_MIN, HEIGHT_LIMIT);
        }
}

class Bush extends Plant {
        private static final double HEIGHT_MIN = 0.01;
        private static final double HEIGHT_LIMIT = 1.5;
        private static final int AGE_LIMIT = 365 * 5;
        private static final int GROWTH_AGE_LIMIT = 365;

        protected boolean hasSpikes_;

        Bush() { this(0, HEIGHT_MIN, false); }

        Bush(int age, double height, boolean hasSpikes)
        {
                super(age, height);
                this.hasSpikes_ = hasSpikes;
```

```
        }

        @Override
        void grow(int daysPassed)
        {
                super.grow(daysPassed);

                super.dead_ = (super.age_ > AGE_LIMIT);

                super.height_ = HEIGHT_MIN + (HEIGHT_LIMIT - HEIGHT_MIN) *
                                ((double)Math.min(super.age_,
        GROWTH_AGE_LIMIT) / GROWTH_AGE_LIMIT);
        }

        @Override
        void printInfo()
        {
                System.out.print("Brush " + (this.hasSpikes_ ? "with" :
        "without") + " spikes, ");
                super.printInfo();
        }

        boolean hasSpikes() { return this.hasSpikes_; }
        void hasSpikes(boolean hasSpikes) { this.hasSpikes_ = hasSpikes; }

        @Override
        void height(double height)
        {
                this.height_ = Math.clamp(height, HEIGHT_MIN, HEIGHT_LIMIT);
        }
}

class Tree extends Plant {
        private static final double HEIGHT_MIN = 0.5;
        private static final double HEIGHT_MAX = 50;
        private static final int CIRCLE_AGE = 10;
        private static final int GROWTH_AGE_LIMIT = 365 * 3;

        protected int circles_ = 0;

        Tree() { this(0, HEIGHT_MIN); }

        Tree(int age, double height)
        {
                super(age, height);
                this.circles_ = super.age_ / CIRCLE_AGE;
        }

        @Override
        void grow(int daysPassed)
        {
                super.grow(daysPassed);
                this.updateCircles();

                double t = Math.min(super.age_, GROWTH_AGE_LIMIT);

                // integral of Math.sin(Math.PI * (t / GROWTH_AGE_LIMIT));
                super.height_ = HEIGHT_MIN + (HEIGHT_MAX - HEIGHT_MIN)
                                        * (1 - Math.cos(Math.PI * (t /
        GROWTH_AGE_LIMIT)));
        }
```

```
        @Override
        void printInfo()
        {
                System.out.print("Tree, Circles: " + this.circles_ + ", ");
                super.printInfo();
        }

        int circles() { return this.circles_; }
        void updateCircles()
        {
                this.circles_ = super.age_ / CIRCLE_AGE;
        }

        @Override
        void height(double height)
        {
                this.height_ = Math.max(height, HEIGHT_MIN);
        }
}
```

Друге завдання:

```
public class Task2 {
        public static void main(String[] args)
        {
                Student[] students = {
                        new Bachelor("Ivanov", "Ivan", "PZ-24-1/9", 1, 55.0),
                        new Bachelor("Petrov", "Petro", "KMP-22-2/9", 2,
        70.0),
                        new Master("Sidorov", "Sidor", "MD-23-1/9", 1, 80.0,
        true, true),
                        new Master("Kuznetsov", "Kuzma", "KMP-23-1/9", 2,
        65.0, false, true),
                        new PhD("Vasilyev", "Vasyl", "MD-25-2/9", 1, 92.0,
        true, true),
                        new PhD("Pavlov", "Pavel", "PZ-23-1/9", 2, 50.0,
        false, false)
                };

                System.out.println("Scholarship Results:");

        System.out.println("-----------------------------------------------------
                for (Student s : students) {
                        System.out.println(s.toString());
                }
        }
}

class Student {
        protected String lastName_;
        protected String firstName_;
        protected String group_;
        protected int studyYear_;
        protected double minScholarship_;
        protected double ratingScore_;

        public Student(String lastName, String firstName, String group, int
        studyYear, double minScholarship, double ratingScore)
```

```
        {
                this.lastName_ = lastName;
                this.firstName_ = firstName;
                this.group_ = group;
                this.studyYear_ = studyYear;
                this.minScholarship_ = minScholarship;
                this.ratingScore_ = ratingScore;
        }

        public double getScholarship()
        {
                return this.minScholarship_ * this.getScholarshipFactor();
        }

        @Override
        public String toString()
        {
                return String.format("%s %s, Group: %s, Year: %d, Score:
        %.2f, Schoolarship: %.2f",
                                this.firstName_, this.lastName_,
                                this.group_, this.studyYear_, this.ratingScore_,
        this.getScholarship());
        }

        protected double getRatingScore()
        {
                return ratingScore_;
        }
        protected double getScholarshipFactor()
        {
                double score = this.getRatingScore();

                if (score <= 60) {
                        return 1;
                } else if (score < 75) {
                        return 1.2;
                } else if (score < 90) {
                        return 1.35;
                } else {
                        return 1.5;
                }
        }
}

class Bachelor extends Student {
        private static final double DEFAULT_MIN_SCHOLARSHIP = 1000;
        public Bachelor(String lastName, String firstName, String group, int
        studyYear, double ratingScore)
        {
                super(lastName, firstName, group, studyYear,
        DEFAULT_MIN_SCHOLARSHIP, ratingScore);
        }
}

class NotBachelor extends Student {
        private boolean hasResearchPapers_;
        private boolean isBudgetPlace_;

        public NotBachelor(String lastName, String firstName, String group,
        int studyYear,
                                double minScholarship, double ratingScore,
```

```
                        boolean hasResearchPapers, boolean isBudgetPlace)
        {
                super(lastName, firstName, group, studyYear, minScholarship,
        ratingScore);
                this.hasResearchPapers_ = hasResearchPapers;
                this.isBudgetPlace_ = isBudgetPlace;
        }

        @Override
        public String toString()
        {
                return super.toString() +
                       (this.isBudgetPlace_ ? ", Budget" : ", Contract") +
                       (this.hasResearchPapers_ ? ", Has research papers" :
        "");
        }

        @Override
        protected double getRatingScore()
        {
                return super.ratingScore_ * (this.hasResearchPapers_ ? 1.1 :
        1.0);
        }

        @Override
        protected double getScholarshipFactor()
        {
                return this.isBudgetPlace_ ? super.getScholarshipFactor() :
        0.0;
        }
}

class Master extends NotBachelor {
        private static final double DEFAULT_MIN_SCHOLARSHIP = 1500;

        public Master(String lastName, String firstName, String group, int
        studyYear,
                      double ratingScore, boolean hasResearchPapers, boolean
        isBudgetPlace)
        {
                super(lastName, firstName, group, studyYear,
        DEFAULT_MIN_SCHOLARSHIP,
                       ratingScore, hasResearchPapers, isBudgetPlace);
        }
}

class PhD extends NotBachelor {
        private static final double DEFAULT_MIN_SCHOLARSHIP = 2000;

        public PhD(String lastName, String firstName, String group, int
        studyYear,
                      double ratingScore, boolean hasResearchPapers, boolean
        isBudgetPlace)
        {
                super(lastName, firstName, group, studyYear,
        DEFAULT_MIN_SCHOLARSHIP,
                       ratingScore, hasResearchPapers, isBudgetPlace);
        }
}
```

Результати тестування:

```
0 days:
Flower of Red color, Age: 0 days, Height: 0.01 meters, Alive
Brush without spikes, Age: 0 days, Height: 0.01 meters, Alive
Tree, Circles: 0, Age: 0 days, Height: 0.5 meters, Alive

100 days:
Flower of Red color, Age: 100 days, Height: 0.25166666666666665 meters, Alive
Brush without spikes, Age: 100 days, Height: 0.4182191780821918 meters, Alive
Tree, Circles: 10, Age: 100 days, Height: 2.5233249469191406 meters, Alive

200 days:
Flower of Red color, Age: 200 days, Height: 0.3 meters, Alive
Brush without spikes, Age: 200 days, Height: 0.8264383561643835 meters, Alive
Tree, Circles: 20, Age: 200 days, Height: 8.427891955724032 meters, Alive

300 days:
Flower of Rotten gray color, Age: 300 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 300 days, Height: 1.2346575342465753 meters, Alive
Tree, Circles: 30, Age: 300 days, Height: 17.73099970400373 meters, Alive

400 days:
Flower of Rotten gray color, Age: 400 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 400 days, Height: 1.5 meters, Alive
Tree, Circles: 40, Age: 400 days, Height: 29.672114454749305 meters, Alive

500 days:
Flower of Rotten gray color, Age: 500 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 500 days, Height: 1.5 meters, Alive
Tree, Circles: 50, Age: 500 days, Height: 43.27504407183115 meters, Alive

600 days:
Flower of Rotten gray color, Age: 600 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 600 days, Height: 1.5 meters, Alive
Tree, Circles: 60, Age: 600 days, Height: 57.4277422180603 meters, Alive

700 days:
Flower of Rotten gray color, Age: 700 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 700 days, Height: 1.5 meters, Alive
Tree, Circles: 70, Age: 700 days, Height: 70.9732187025082 meters, Alive

800 days:
Flower of Rotten gray color, Age: 800 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 800 days, Height: 1.5 meters, Alive
Tree, Circles: 80, Age: 800 days, Height: 82.80412401047222 meters, Alive

900 days:
Flower of Rotten gray color, Age: 900 days, Height: 0.3 meters, Dead
Brush without spikes, Age: 900 days, Height: 1.5 meters, Alive
Tree, Circles: 90, Age: 900 days, Height: 91.95327568319513 meters, Alive
```

Рисунок 1 - Симуляція росту різних рослин

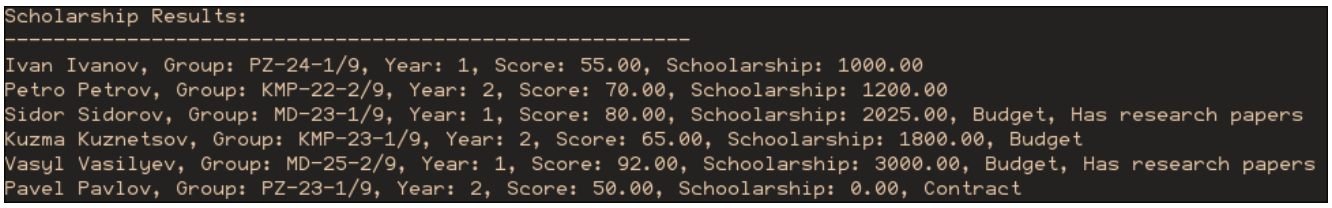


Рисунок 2 - Завдання 2

Контрольні запитання

1. Яким чином у різних мовах програмування поділяють характер взаємних відносин між суперкласом та підкласом успадкування.

У C# та Java доступно тільки одиночне успадкування, але можлива реалізація набору інтерфейсів

У таких мовах, як C++ та Python множинне успадкування можливе, і немає чіткої межі між абстрактним класом та інтерфейсом.

2. Розкрийте зміст властивості, яка отримала назву “пошук реалізації”.

Пошук реалізації — пошук середовищем виконання правильного методу для виклику із урахуванням поліморфізму.

Пошук починається із класу об’єкту і, якщо він містить метод, береться найнижче в ієрархії його перевизначення. Інакше пошук продовжується у класі вище по ієрархії.

3. Яким чином можна заблокувати успадкування для окремих класів та при перевизначенні окремих методів.

Ключове слово final робить клас чи його окреме поле недоступним для перевизначення

4. У якому випадку члени класу належать самому суперкласові та не беруть участі в механізмі успадкування.

Якщо вони оголошені як private та/або static

5. Яку особливість має успадкування статичних членів класу.

Статичні поля спільні для всієї ієрархії, при перевизначені не будуть динамічно зв’язуватися.

6. Чи може міститися абстрактний метод у звичайному (не абстрактному) класі?

Якщо клас не визначено як abstract він усе одно може мати абстрактні методи

7. Яким чином включаються члени суперкласу до підкласу, якщо вони визначені за умовчанням, чи можна до них звертатися просто на ім’я?

За завмовчанням поля є friendly, що означає що вони будуть успадковуватися із доступом із підкласу у межах пакету.

8. Яким чином включаються члени суперкласу до підкласу, якщо вони визначені як приватні?

Вони будуть присутні у пам’яті, але не доступні із коду того класу, що їх успадкував

9. Як створюється об’єкт підкласу, якщо конструктори оголошені як у підкласі, так і у суперкласі?

Спочатку викликається конструктор суперкласу (без параметрів якщо не викликано явно), вже після - підкласу.

10. До якого рівня ієрархії відбувається звернення у конструкторі підкласу, якщо в ньому є виклик конструктора super()?

Виклик super() завжди викликає безпосереднього попередника даного класу